

PROJE-3: Akıllı Veri Analitiği ve Makine Öğrenmesi Uygulaması

Bulut Bilişim Dersi (3522) — Proje Raporu

Öğrenci: Arife Ebrar Üstüner | **Tarih:** Mayıs 2026 | **Platform:** AWS (S3, Lambda, API Gateway)

Canlı Web Sitesi: <http://bulut-proje-titanic-ml-titanic-ebrar.s3-website.eu-central-1.amazonaws.com/>

Sunum Videosu:

<https://drive.google.com/drive/u/0/folders/1arGKnutvXdRjU1hN1j7SE6bGzDSDoRUp>

API Endpoint: <https://l7om4ms6bi.execute-api.eu-central-1.amazonaws.com/default/titanic-predict>

1. Proje Özeti

Bu projede, Titanic gemisindeki yolcuların hayatta kalıp kalmayacağını tahmin eden bir makine öğrenmesi modeli geliştirilmiş ve AWS bulut platformuna dağıtılmıştır.

Hedefler:

- Gerçek bir veri seti üzerinde makine öğrenmesi modeli geliştirmek
- Modeli bir REST API'ye dönüştürmek
- API'yi AWS Lambda ile serverless (sunucusuz) olarak buluta taşımak
- Web arayüzü ile son kullanıcıya sunmak

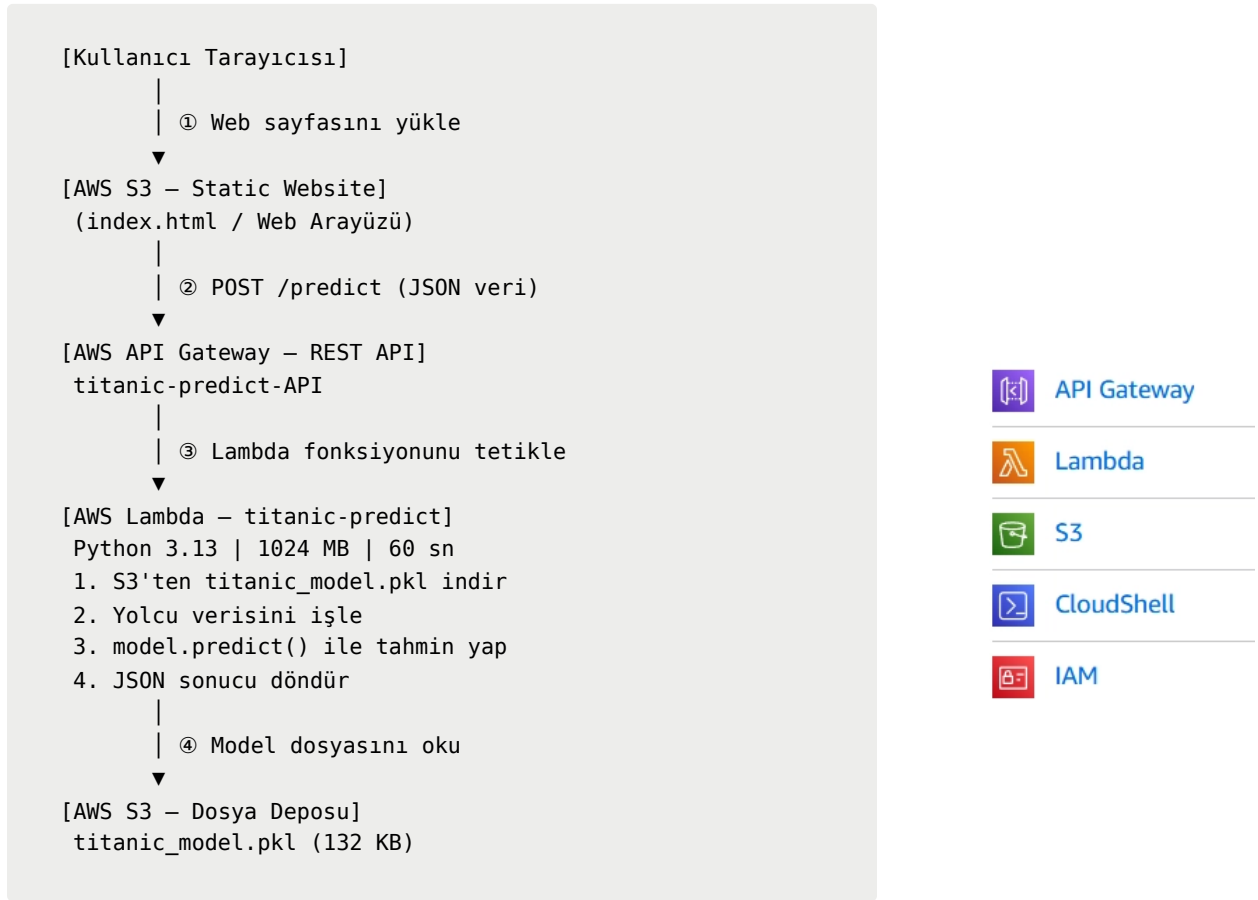
Sonuç: Kullanıcı bir Titanic yolcusunun bilgilerini girdiğinde (sınıf, cinsiyet, yaş, ücret vb.), sistem %80.3 doğrulukla hayatta kalıp kalmayacağını tahmin etmektedir.

2. Kullanılan Teknolojiler

Kategori	Teknoloji	Amaç
Backend Dili	Python 3.13	Model eğitimi ve API
ML Kütüphanesi	Scikit-learn	Makine öğrenmesi
Veri İşleme	Pandas, NumPy	Veri manipülasyonu
Görselleştirme	Matplotlib, Seaborn	Grafik oluşturma

Kategori	Teknoloji	Amaç
Yerel API	Flask	Geliştirme ortamı testi
API Test	Postman	API endpoint testi
Bulut (Depolama)	AWS S3	Model ve web sitesi barındırma
Bulut (Hesaplama)	AWS Lambda	Serverless fonksiyon
Bulut (API)	AWS API Gateway	HTTP endpoint
Versiyon Kontrol	Git & GitHub	Kod yönetimi

3. Sistem Mimarisi



4. Uygulama Süreci (Gün Gün)

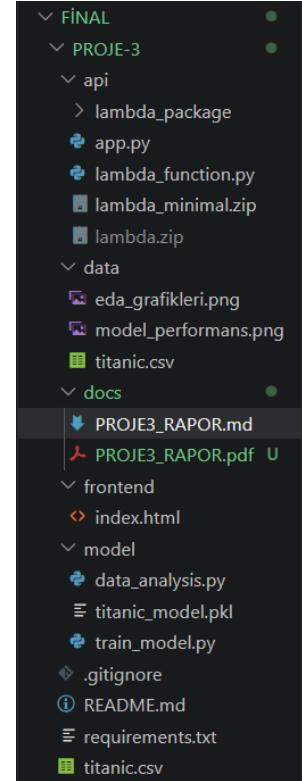
Gün 1 — 24 Mayıs 2026: Hazırlık ve Temel Kodlar

4.1.1 Proje Klasör Yapısının Oluşturulması

Proje başlangıcında, kodların düzenli tutulması için aşağıdaki klasör yapısı oluşturulmuştur:

```
FINAL/
├── PROJE-3/
│   ├── data/          → Ham veri ve grafikler
│   ├── model/         → ML kodu ve eğitilmiş model
│   ├── api/           → Flask ve Lambda kodları
│   ├── frontend/      → Web arayüzü
│   └── docs/          → Rapor
├── PROJE-4/          → (İlerleyen haftalarda)
└── PROJE-6/          → (İlerleyen haftalarda)
```

Bu yapı, her bileşenin (veri, model, API, arayüz) birbirinden ayrı tutulmasını sağlamaktadır. Profesyonel yazılım projelerinde bileşenlerin birbirinden bağımsız olması, bakım ve hata ayıklamayı kolaylaştırmaktadır.



4.1.2 Gereksinim Dosyası (requirements.txt)

Python kütüphaneleri `requirements.txt` dosyasına kaydedilmiştir. Bu dosya, projenin başka bir ortamda kurulabilmesi için gerekli tüm bağımlılıkları listeler:

```
scikit-learn pandas numpy matplotlib seaborn flask joblib boto3
```

(Not: Ortam kurulumlarında sürüm çakışması yaşanmaması için versiyon kısıtlamaları bilerek kaldırılmıştır.)

`pip install -r requirements.txt` komutu ile tüm kütüphaneler tek seferde kurulabilir.

4.1.3 İlk Git Commit

Günün sonunda tüm dosyalar Git ile versiyon kontrolüne alınmış ve GitHub'a yüklenmiştir.

```
git add .  
git commit -m "feat: PROJE-3 klasör yapısı oluşturuldu"  
git push
```

Gün 2 — 27 Mayıs 2026: EDA, Model Eğitimi ve Yerel Test

4.2.1 Veri Seti

Kullanılan Veri Seti: Titanic - Machine Learning from Disaster

Kaynak: Stanford Üniversitesi açık veri seti —

<https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv>

Veri seti 887 yolcunun bilgisini içermektedir:

Sütun	Açıklama	Örnek Değer
Survived	Hayatta kaldı mı? (0/1)	0 veya 1
Pclass	Yolcu sınıfı	1, 2 veya 3
Sex	Cinsiyet	male / female
Age	Yaş	22.0
Siblings/Spouses Aboard	Kardeş/eş sayısı	1
Parents/Children Aboard	Ebeveyn/çocuk sayısı	0
Fare	Bilet ücreti (\$)	7.25

4.2.2 EDA (Keşifsel Veri Analizi)

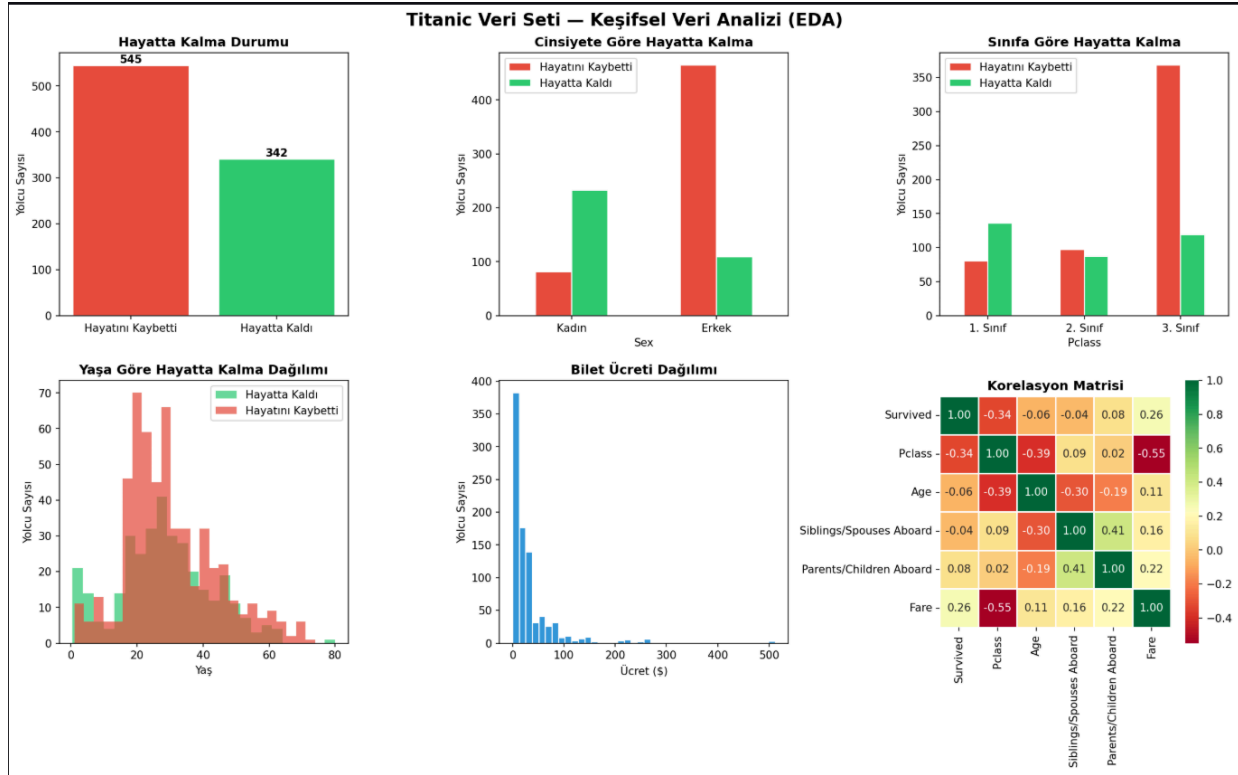
EDA (Exploratory Data Analysis), makine öğrenmesi modelinden önce veriyi tanıma ve anlama sürecidir. `model/data_analysis.py` scripti çalıştırılarak aşağıdaki bulgular elde edilmiştir:

Genel İstatistikler: Toplam yolcu: 887 | Hayatta kalan: 342 (%38.6) | Hayatını kaybeden: 545 (%61.4) | Eksik değer: Yok.

Temel Bulgular:

- Kadın yolcuların büyük çoğunluğu hayatta kalmıştır

- 1. sınıf yolcular 3. sınıfa göre çok daha yüksek hayatta kalma oranına sahiptir
- Genç yolcular ve çocuklar daha yüksek hayatta kalma oranı göstermektedir



4.2.3 Veri Ön İşleme (Preprocessing)

CSV dosyamızda cinsiyet bilgisi "male" ve "female" olarak metin şeklinde yer almaktadır. Ancak **makine öğrenmesi algoritmaları metinleri okuyamaz, yalnızca sayısal verilerle çalışabilir**. Bu sebeple cinsiyet bilgisini sayısal formata (`male=0` , `female=1`) çevirdik. Bu işleme **Label Encoding** denir.

Uyguladığımız diğer ön işleme adımları:

- **Eksik Değerler:** Yaş ve ücret sütunlarındaki boşluklar medyan değerle dolduruldu.
- **Biniş Limanı:** C=0, Q=1, S=2 olarak sayısallaştırıldı.
- **Yeni Özellik:** SibSp ve Parch sütunları toplanarak `FamilySize` oluşturuldu.
- **Gereksizler:** Name, PassengerId gibi modele katkısı olmayan sütunlar çıkarıldı.

4.2.4 Model Eğitimi

`model/train_model.py` scripti ile üç farklı makine öğrenmesi algoritması 5 katlı çapraz doğrulama (cross-validation) ile karşılaştırılmıştır:

Logistic Regression (Lojistik Regresyon): Özelliklerin ağırlıklı toplamına dayalı doğrusal bir sınıflandırma yöntemidir. En basit modeldir; yorumlanması kolaydır ancak karmaşık örüntüleri yakalamada sınırlıdır.

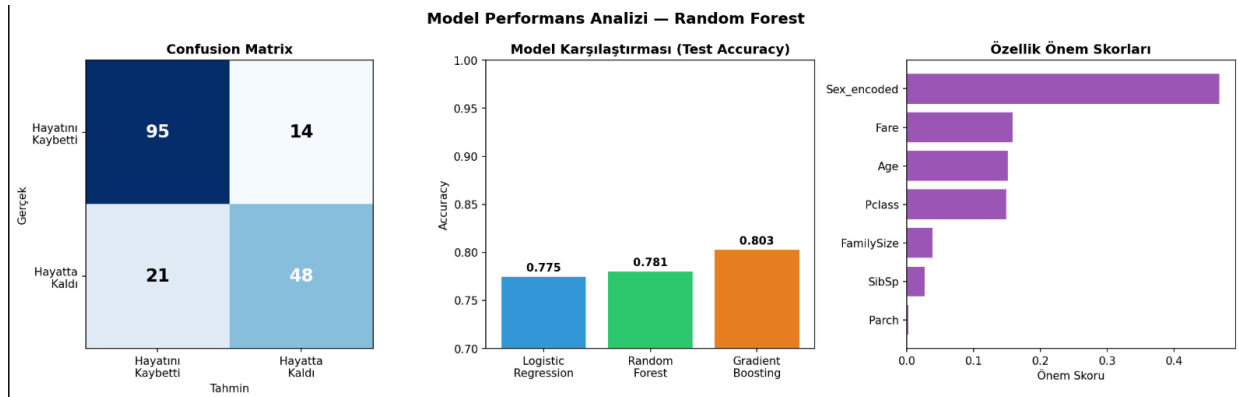
Random Forest (Rastgele Orman): 100 karar ağacının oluşturduğu bir topluluk modelidir. Her ağaç bağımsız tahmin yapar; çoğunluğun kararı final tahmin olur. Tek bir ağaca göre çok daha güvenilirdir.

Gradient Boosting (Gradyan Artırma): Her adımda bir önceki modelin hatalarını öğrenerek düzelteren bir yöntemdir. Zayıf modellerden güçlü bir model oluşturur.

Model Karşılaştırma Sonuçları:

Model	CV Accuracy	Test Accuracy
Logistic Regression	%80.5 ± 2.9	%77.5
Random Forest	%81.4 ± 1.8	%78.1
Gradient Boosting	%83.4 ± 4.0	%80.3

Kazanan: Gradient Boosting — %80.3 Test Accuracy



Eğitim sonunda model `model/titanic_model.pkl` dosyasına kaydedilmiştir. `.pkl` (pickle) formatı Python'da nesneleri dosyaya kaydetmek için kullanılan standarttır. Model bir kez eğitilip kaydedilir; her tahmin için yeniden eğitilmesine gerek kalmaz.

4.2.5 Flask API ile Yerel Test

Eğitilen model bir REST API'ye dönüştürülmüştür. `api/app.py` dosyası Flask web framework'ü kullanılarak yazılmıştır.

API Endpoint'leri: `GET /` (API bilgileri) | `GET /health` (sağlık kontrolü) | `POST /predict` (tahmin yap)

Postman ile Test: Method: POST | URL: `http://localhost:5000/predict`

```
{"Pclass": 3, "Sex": "male", "Age": 25, "SibSp": 0, "Parch": 0, "Fare": 7.25, "Embarked": "S"}
```

API'nin döndürdüğü cevap:

```
{"hayatta_kaldi": false, "mesaj": "✗ Hayatını kaybederdi.", "olasilik": {"hayatini_kaybetti": 0.9189, "hayatta_kaldi": 0.0811}, "tahmin": 0}
```

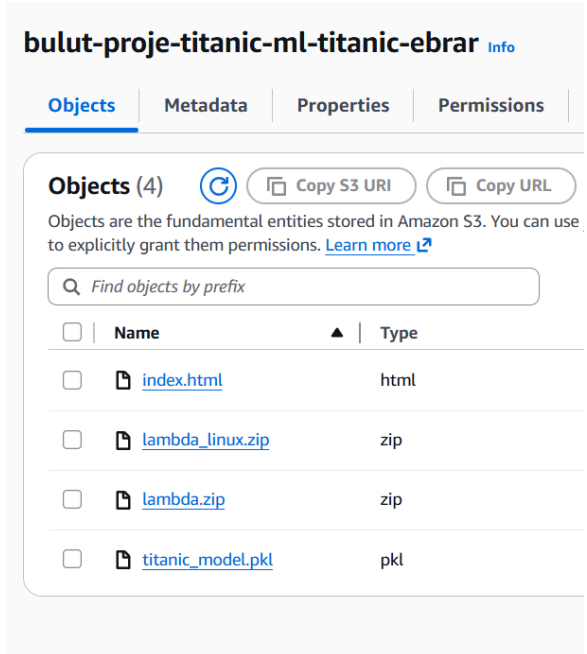
Bu cevap, 25 yaşında 3. sınıf erkek yolcunun %91.89 ihtimalle hayatını kaybedeceğini göstermektedir — tarihi verilerle örtüşmektedir.

Gün 3 — 28 Mayıs 2026: AWS Dağıtımı ve Canlı Yayın

4.3.1 AWS S3 Bucket Oluşturma

AWS S3 bu projede iki amaçla kullanılmıştır: (1) `titanic_model.pkl` depolamak — Lambda her çalıştığında buradan okur, (2) `index.html` dosyasını Static Website Hosting ile web sitesi olarak yayınlamak.

Bucket: `bulut-proje-titanic-ml-titanic-ebrar` | Bölge: eu-central-1 (Frankfurt) | Yüklenen: `titanic_model.pkl` (132 KB), `lambda_linux.zip` (80 MB)



4.3.2 AWS Lambda Fonksiyonu

AWS Lambda, sunucu kurmadan kod çalıştırmayı sağlayan Serverless Computing hizmetidir. İstek geldiğinde fonksiyon uyanır, işlemi yapar, döner ve uyur. Kullanılmayan süre için ücret ödenmez (Free Tier: aylık 1 milyon istek ücretsiz).

● Karşılaşılan Hata 1 — Windows/Linux Kütüphane Uyumsuzluğu:

Python kütüphaneleri Windows işletim sisteminde derlendiği için Lambda'nın Linux ortamında çalışmaz ve şu hata alınır:

```
AttributeError: module 'os' has no attribute 'add_dll_directory'
```

✓ **Çözüm:** AWS CloudShell kullanıldı. CloudShell, AWS'nin kendi tarayıcı tabanlı Linux terminalidir. Kütüphaneler burada (Linux'ta) derlendiğinde Lambda ile tamamen uyumlu hale gelir.

```
mkdir lambda_pkg
pip install scikit-learn numpy joblib -t lambda_pkg/
zip -r lambda_linux.zip lambda_pkg/
aws s3 cp lambda_linux.zip s3://bulut-proje-titanic-ml-titanic-ebrar/
aws lambda update-function-code \
  --function-name titanic-predict \
  --s3-bucket bulut-proje-titanic-ml-titanic-ebrar \
  --s3-key lambda_linux.zip
```

● Karşılaşılan Hata 2 — Lambda Timeout:

Lambda'nın varsayılan zaman aşımı 3 saniyedir. Scikit-learn kütüphanesi ve model dosyasının S3'ten yüklenmesi daha uzun sürdüğünden test sırasında şu hata alındı:

```
Task timed out after 3.00 seconds
```

✓ **Çözüm:** Lambda Configuration → General Configuration → Timeout değeri **60 saniyeye** çıkarıldı.

● Karşılaşılan Hata 3 — Lambda Yetersiz Bellek:

Scikit-learn kütüphanesini yüklemek için 128 MB varsayılan bellek yetersiz geldi, Lambda çakıldı.

✓ **Çözüm:** Memory değeri **1024 MB'a** çıkarıldı.

Lambda Ayarları:

Ayar	Değer
Fonksiyon Adı	titanic-predict
Runtime	Python 3.13
Bellek	1024 MB
Timeout	60 saniye
Handler	lambda_function.lambda_handler

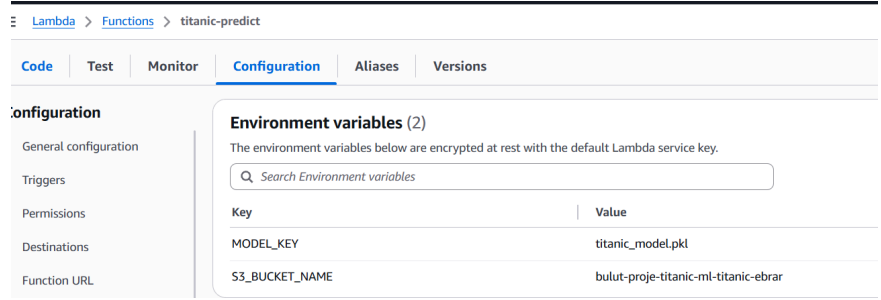
Environment Variables: `S3_BUCKET_NAME` = bulut-proje-titanic-ml-titanic-ebrar |
`MODEL_KEY` = titanic_model.pkl

IAM İzinleri:

Lambda'nın S3'teki model dosyasını okuyabilmesi için

`AmazonS3ReadOnlyAccess`

politikası eklenmiştir.



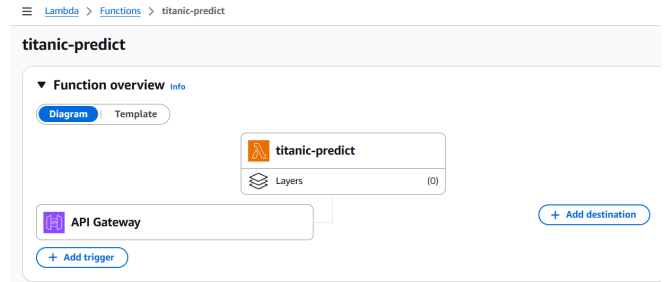
Lambda Test Sonucu: Lambda'nın Test sekmesinde yapılan testte 1. sınıf kadın yolcu için %96.15 hayatta kalma olasılığı döndü — tarihi verilerle tam örtüşmektedir.

4.3.3 AWS API Gateway

API Gateway, Lambda fonksiyonuna internet üzerinden erişilebilir bir HTTP URL oluşturur; Lambda'nın "kapıcısı" rolünü üstlenir.

Lambda → **Add trigger** → API Gateway → REST API → Open → Add

Oluşturulan API: `titanic-predict-API` | Method: ANY | Auth: NONE



Endpoint: `https://l7om4ms6bi.execute-api.eu-central-1.amazonaws.com/default/titanic-predict`

4.3.4 CORS Nedir? Neden Gerekli?

● Karşılaşılan Hata 4 — CORS Hatası:

Web sitesi (S3) ve API farklı alan adlarında (farklı "origin") bulunmaktadır. Tarayıcılar güvenlik nedeniyle varsayılan olarak farklı kaynaklara istek atmayı engeller. Bu yüzden web sitesinden API'ye istek gönderildiğinde tarayıcı blokluyordu.

```
Access to fetch at 'https://l7om4ms6bi...' from origin 'http://bulut-proje-...s3-websit
e...'
has been blocked by CORS policy
```

✓ **Çözüm:** API Gateway'de CORS etkinleştirildi. Bu sayede API, yanıt başlıklarına `Access-Control-Allow-Origin: *` ekler ve tarayıcıya "Her kaynaktan gelen istek kabul edilir" mesajını verir.

4.3.5 S3 Static Website Hosting

Web arayüzü (`frontend/index.html`) AWS S3 üzerinde barındırılmıştır. S3 Static Website Hosting, bir sunucuya ihtiyaç duymadan web sayfası yayınlamayı sağlar.

● Karşılaşılan Hata 5 — S3 Erişim Reddi:

S3 varsayılan olarak tüm genel erişimi engeller. `index.html` yüklenip URL'ye girildiğinde "Access Denied" hatası alındı.

✓ **Çözüm:** Bucket → Permissions → Block public access **kapatıldı** ve Bucket policy ile herkese okuma izni verildi:

```
{"Effect": "Allow", "Principal": "*", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::
bulut-proje-titanic-ml-titanic-ebrar/*"}
```

Yapılan Ayarlar: Properties → Static website hosting → Enable | Index document: `index.html`

PROJE-3 — BULUT BİLİŞİM DERSİ

Titanic Hayatta Kalma Tahmini

AWS Lambda + Scikit-learn (Random Forest) — Makine Öğrenmesi API

Yolcu Bilgileri

YOLCU SINIFI (PCLASS)

3. Sınıf (Third Class)

CİNSİYET (SEX)

Erkek (Male)

YAŞ (AGE)

25

KARDEŞ / EŞ SAYISI (SIBSP)

0

EBEVEYN / ÇOCUK SAYISI (PARCH)

0

BİLET ÜCRETİ — \$ (FARE)

7.25

BİNİŞ LİMANI (EMBARKED)

Southampton (S)

Tahmin Et

Proje Bilgisi

Backend Python 3.11

ML Kütüphane Scikit-learn

Model Random Forest

Veri Seti Titanic (891 kayıt)

Bulut AWS Lambda + S3

API API Gateway (REST)

Frontend Host AWS S3 Static

Test Accuracy ~%83

Nasıl Çalışır?

Girdiğiniz yolcu bilgileri API Gateway üzerinden AWS Lambda'ya iletilir. Lambda, S3 üzerindeki eğitilmiş model yükler ve tahmin yaparak sonucu döndürür.

PROJE-3 — Bulut Bilgi Dersi (3522) | AWS Lambda + Scikit-learn | Titanic Hayatta Kalma Tahmini

PROJE-3 — BULUT BİLİŞİM DERSİ

Titanic Hayatta Kalma Tahmini

AWS Lambda + Scikit-learn (Random Forest) — Makine Öğrenmesi API

Yolcu Bilgileri

YOLCU SINIFI (PCLASS)

3. Sınıf (Third Class)

CİNSİYET (SEX)

Erkek (Male)

YAŞ (AGE)

25

KARDEŞ / EŞ SAYISI (SIBSP)

0

EBEVEYN / ÇOCUK SAYISI (PARCH)

0

BİLET ÜCRETİ — \$ (FARE)

7.25

BİNİŞ LİMANI (EMBARKED)

Southampton (S)

Tekrar Tahmin Et

Proje Bilgisi

Backend Python 3.11

ML Kütüphane Scikit-learn

Model Random Forest

Veri Seti Titanic (891 kayıt)

Bulut AWS Lambda + S3

API API Gateway (REST)

Frontend Host AWS S3 Static

Test Accuracy ~%83

Nasıl Çalışır?

Girdiğiniz yolcu bilgileri API Gateway üzerinden AWS Lambda'ya iletilir. Lambda, S3 üzerindeki eğitilmiş model yükler ve tahmin yaparak sonucu döndürür.

Tahmin Sonucu

Hayatını Kaybetti

Hayatını kaybetti.

Hayatta Kalma 8.1%

Hayatını Kaybetme 91.9%

PROJE-3 — Bulut Bilgi Dersi (3522) | AWS Lambda + Scikit-learn | Titanic Hayatta Kalma Tahmini

5. Makine Öğrenmesi Modeli Detayları

Performans Metrikleri

Metrik	Değer	Açıklama
Accuracy	%80.3	Tüm tahminlerin doğruluk oranı
Precision	%77.4	"Hayatta kaldı" tahminlerinde isabet oranı
Recall	%69.6	Gerçek hayatta kalanların tespit oranı
F1-Score	%73.3	Precision ve Recall'un harmonik ortalaması

Confusion Matrix

	Tahmin: Hayatını Kaybetti	Tahmin: Hayatta Kaldı
Gerçek: Hayatını Kaybetti	95  (Doğru)	14  (Yanlış)
Gerçek: Hayatta Kaldı	21  (Yanlış)	48  (Doğru)

En Etkili Özellikler (Feature Importance)

Gradient Boosting modeline göre hayatta kalmayı etkileyen en önemli faktörler:

1. **Cinsiyet** — en belirleyici faktör
2. **Bilet Ücreti** — sosyoekonomik durumu yansıtır
3. **Yaş** — özellikle çocuklar önceliklendirilmiştir
4. **Yolcu Sınıfı** — 1. sınıf en yüksek hayatta kalma oranına sahip

6. API Kullanımı

İstek Formatı:

```
POST https://l7om4ms6bi.execute-api.eu-central-1.amazonaws.com/default/titanic-predict

{"Pclass": 3, "Sex": "male", "Age": 25, "SibSp": 0, "Parch": 0, "Fare": 7.25, "Embarked": "S"}
```

Cevap Formatı:

```
{"tahmin": 0, "hayatta_kaldi": false, "olasilik": {"hayatini_kaybetti": 0.9189, "hayatta_kaldi": 0.0811}, "mesaj": "❌ Hayatını kaybederdi."}
```

7. Sonuç ve Değerlendirme

Karşılaşılan Zorluklar ve Çözümler:

#	Sorun	Hata Mesajı	Çözüm
1	Windows/Linux kütüphane uyumsuzluğu	<code>os.add_dll_directory</code> hatası	AWS CloudShell ile Linux'ta derleme
2	Lambda zaman aşımı	Task timed out after 3.00 seconds	Timeout 3 sn → 60 sn
3	Lambda bellek yetersizliği	Lambda crash	RAM 128 MB → 1024 MB
4	CORS hatası	CORS policy blocked	API Gateway'de CORS etkinleştirildi
5	S3 erişim reddi	Access Denied	Bucket policy + public access açıldı

Teknik Kazanımlar: Gerçek veri seti üzerinde EDA ve veri ön işleme | Üç farklı ML algoritmasının karşılaştırmalı değerlendirmesi | REST API geliştirme (Flask) | Serverless mimari (AWS Lambda) | Nesne depolama (AWS S3) | API yönetimi (AWS API Gateway) | Linux ortamında paket derleme (CloudShell) | CORS güvenlik mekanizması.

Sonuç: Geliştirilen sistem Titanic yolcularının hayatta kalıp kalmayacağını %80.3 doğrulukla tahmin etmektedir. Model AWS Lambda üzerinde serverless olarak çalışmakta, API Gateway aracılığıyla internet üzerinden erişilebilmekte ve S3 üzerinde barındırılan web arayüzü ile kullanıcılara sunulmaktadır.

Rapor Tarihi: 28 Mayıs 2026

GitHub: <https://github.com/Ebrar3/Bulut-proje/tree/main/FİNAL/PROJE-3>